

RESEARCH PAPER

SmoIVLM

Redefining Small and Efficient
Multimodal Models

Compact Vision-Language Models for Edge Deployment

0.8 GB

Min RAM Usage

300x

Smaller than Idefics-80B

59.8%

Avg Score (2.2B)

Large VLMs Are Powerful but Impractical for Edge Deployment



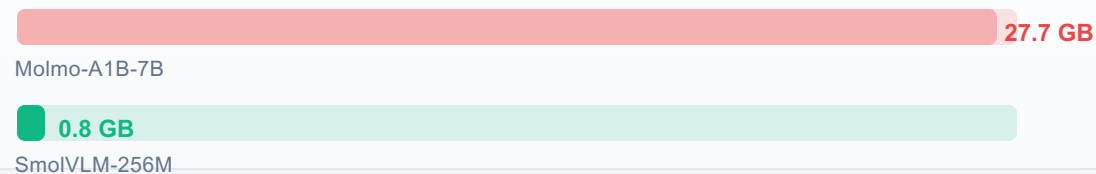
The Edge Deployment Gap

- State-of-the-art VLMs require massive GPU memory
- Molmo-A1B-7B needs 27.7 GB RAM for inference
- Idefics-80B has 80 billion parameters
- Mobile devices typically have 4-16 GB shared RAM

Core Tension

Cloud-dependent VLMs cannot serve privacy-sensitive or latency-critical applications on mobile/edge devices.

RAM Requirement Comparison



SmolVLM: Designed for Edge

256M

0.8 GB RAM

44.0% avg
sub-1GB**500M**

1.2 GB RAM

51.0% avg
on-device**2.2B**

4.9 GB RAM

59.8% avg
competitive

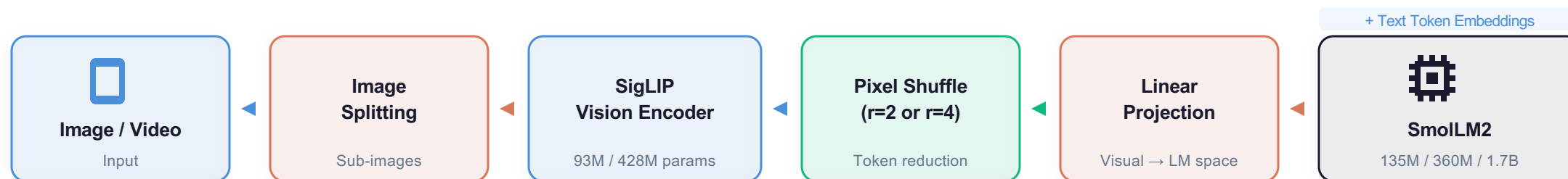
Key Result:

SmolVLM-256M surpasses the 300x larger Idefics-80B

Three Architectural Innovations:

- Balanced encoder-LM compute allocation
- Aggressive pixel shuffle token compression
- Extended context length (2K to 16K tokens)

SmoIVLM Architecture: Image Splitting, Pixel Shuffle, and SmoLM2 Backbone



1

Image Splitting

Pre-processing for high-resolution inputs

Images are divided into sub-images to handle high-resolution inputs within the fixed context window of the LM.

Video frames are sampled and processed similarly, enabling multi-frame reasoning.

Pre-process

2

Encode & Compress

SigLIP + Pixel Shuffle pipeline

SigLIP encoder extracts visual features. Pixel shuffle (space-to-depth) rearranges spatial features into channels, reducing visual tokens by r^2 (4x or 16x).

Linear projection maps visual tokens into the LM input space.

Compress

3

SmoLM2 Backbone

Language model fusion

Compressed visual tokens are concatenated with text token embeddings and fed into the SmoLM2 backbone.

Three sizes: 135M, 360M, 1.7B LM
Context: 8K (256M/500M), 16K (2.2B)

Generate

FINDING 1

Smaller Vision Encoders Outperform Larger Ones in Compact VLMs

Encoder Performance Comparison with 135M LM



Key Insight

For compact VLMs, a smaller 93M encoder outperforms the 428M encoder when paired with a 135M language model.

Why?

Balanced compute allocation matters more than raw encoder size.

Scale Crossover Point

Only at 1.7B LM scale does the larger 428M SigLIP-SO400M justify its 10% parameter overhead.

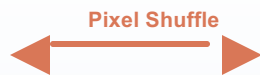
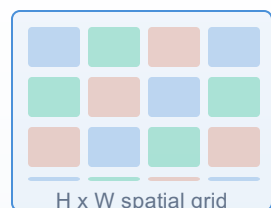
Implication:

Don't over-provision the encoder for small LMs.

Aggressive Pixel Shuffle (r=4) Cuts Visual Tokens 16x for Small Models

Pixel Shuffle Mechanism (Space-to-Depth)

Before: Spatial Features



After: Compressed Tokens



$(H/r) \times (W/r)$ tokens, r^2 channels

16x

Token Reduction at r=4

4x4 spatial block $\rightarrow$ 1 token with 16 channels

r=4

Small VLMs (256M / 500M)

Recommended: Aggressive Compression

16x token reduction eases attention overhead
Improves long-context modeling capability
Frees capacity for the language model backbone

Best for edge

r=2

Larger VLMs (2.2B)

Moderate Compression Preferred

4x token reduction preserves spatial detail
Larger LMs have capacity for more visual tokens
Preserves fine-grained visual information

Best for quality

Extending Context from 2K to 16K Tokens Unlocks Higher Image Resolution



Context Extension

Performance gains from longer sequences

2K tokens

Limited resolution

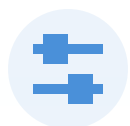
16K tokens

High resolution

Significant performance improvement observed across all model sizes.

8x more context = higher image quality

More tokens for visual detail

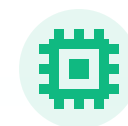


Technical Approach

How context extension is achieved

- 1 RoPE Base Adjustment**
10K → 273K
- 2 Fine-tuning on Mixed Data**
Long + short-context samples
- 3 Gradual Extension**
2K → 8K → 16K tokens

RoPE base 273K enables stable position encoding at 16K length



Configuration

Context length per model size

SmoIVLM-256M

8K context

SmoIVLM-500M

8K context

SmoIVLM-2.2B

16K context

Largest model uses full 16K for maximum image resolution support

SmoIVLM Achieves Competitive Performance at Sub-1GB to 4.9GB RAM

Capability	Benchmark	SmoIVLM 256M	SmoIVLM 500M	SmoIVLM 2.2B	Best Efficient OS
Single-Image	OCRBench	52.6%	61.0%	72.9%	54.7% (Molmo)
Single-Image	AI2D	46.4%	59.2%	70.0%	71.0% (Molmo)
Single-Image	ChartQA	55.6%	62.8%	68.7%	48.0% (Molmo)
Single-Image	TextVQA	50.2%	60.2%	73.0%	61.5% (Molmo)
Single-Image	DocVQA	58.3%	70.5%	80.0%	77.7% (Molmo)
Single-Image	ScienceQA	73.8%	80.0%	89.6%	87.5% (Molmo)
Multi-task	MMMU	29.0%	33.7%	42.0%	33.9% (Molmo)
Multi-task	MathVista	35.9%	40.1%	51.5%	37.6% (Molmo)
Multi-task	MMStar	34.6%	38.3%	46.0%	43.1% (Molmo)
Video	Video-MME	33.7%	42.2%	52.1%	45.0% (InternVL2)
Video	MLVU	40.6%	47.3%	55.2%	48.2% (InternVL2)
Average	All Benchmarks	44.0%	51.0%	59.8%	--

RAM Usage
(batch=1)

0.8 GB

SmoIVLM-256M

1.2 GB

SmoIVLM-500M

4.9 GB

SmoIVLM-2.2B

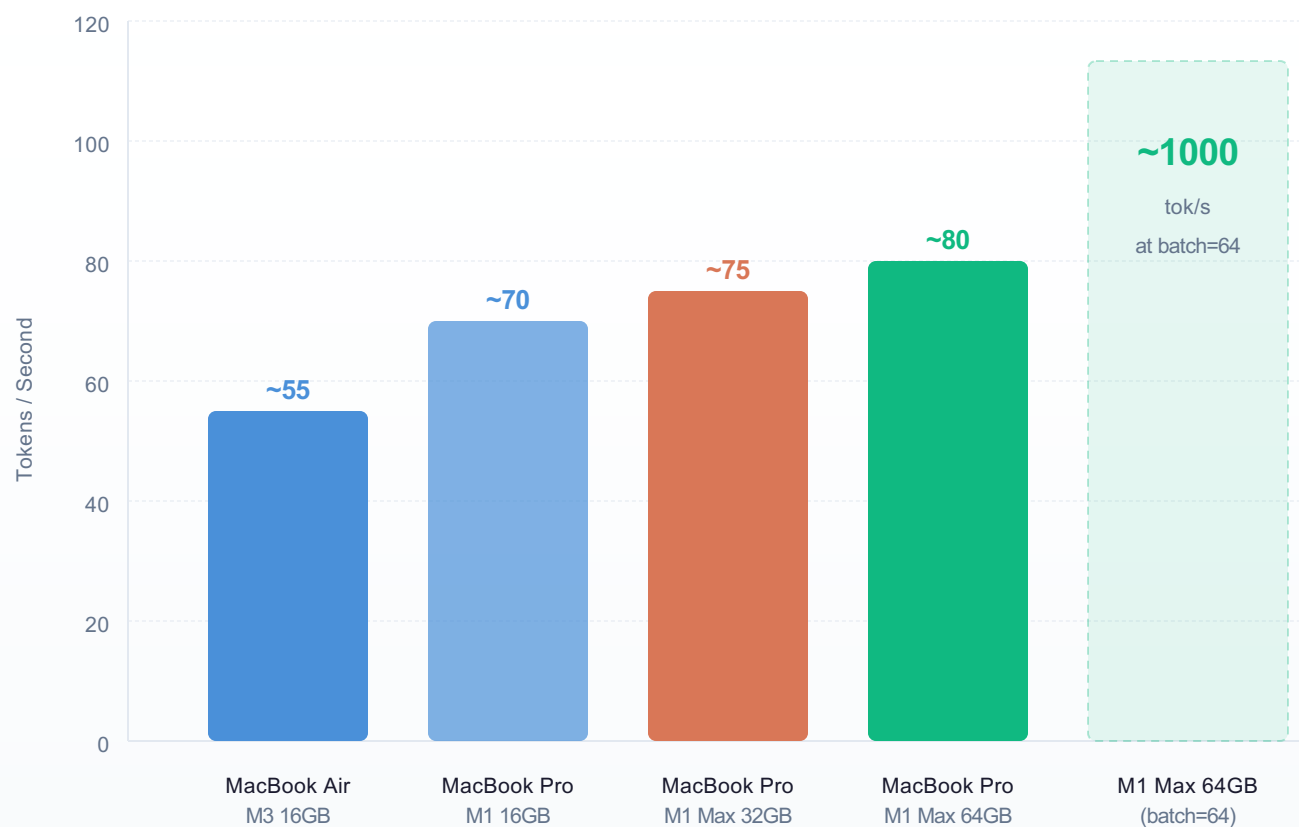
27.7 GB

Molmo-A1B-7B

SmoIVLM-2.2B rivals SOTA models at half the GPU memory
SmoIVLM-256M (0.8 GB) surpasses the 300x larger Idefics-80B

Real-Time Inference on iPhones: Up to 80 Tokens/Second on Apple Silicon

Throughput by Device (SmoIVLM-256M, batch=1)



HuggingSnap iOS App

- Fully local processing, no cloud dependency
- Real-time photo and video analysis
- Privacy-first: data never leaves device
- Throughput scales linearly with batch size

80

tok/s peak

Pro M1 Max 64GB

batch=1

55

tok/s on laptop

MacBook Air M3

16GB RAM

Implication for On-Device AI

SmoIVLM-256M delivers real-time VLM inference on consumer hardware with zero cloud dependency.

Key Takeaways: Design Principles for Efficient Multimodal Models

1 Balance Encoder-LM Compute

Smaller vision encoders serve compact VLMs better. The 93M SigLIP-B/16 outperforms 428M SigLIP-SO400M for small LMs.

2 Extend Context Length

Extending from 2K to 16K tokens unlocks higher image resolution and significant performance gains via RoPE base adjustment (10K $\rightarrow$ 273K).

3 Aggressive Token Compression

Pixel shuffle at $r=4$ gives 16x token reduction for small models, easing attention overhead and improving long-context modeling.

4 Sub-1GB Inference Is Achievable

SmoVLM-256M runs in only 0.8 GB GPU RAM, enabling deployment on smartphones and low-power edge devices with real-time throughput (~ 80 tok/s on Apple Silicon).

5 Fully Open-Source Release

All model weights, training datasets, code, and the HuggingSnap mobile app are fully open-sourced to promote reproducibility and on-device AI research.

Architectural efficiency matters more than raw scale for edge deployment.